
Software Requirements Specification and Design

for

Delhi Power Demand Predictor

Version 1.0 approved

Prepared by PowerRangers

- **Aman Raj Kushwaha - 24293916128**
- **Kartik Nath Khatri - 24293916108**
 - **Piyush Sharma - 24293916133**
 - **Navya Rawat - 24293916074**
 - **Gaurav Rao - 24293916091**

SLDC

26 Jan 2026

Table of Contents

1. Introduction	4
1.1. Purpose	4
1.2. Document Conventions	4
1.3. Intended Audience and Reading Suggestions	4
1.4. Product Scope	4
1.5. References	5
2. Overall Description	5
2.1. Product Perspective	5
2.2. Product Functions	5
2.3. User Classes and Characteristics	6
2.4. Operating Environment	6
2.5. Design and Implementation Constraints	6
2.6. User Documentation	6
2.7. Assumptions and Dependencies	6
3. External Interface Requirements	7
3.1. User Interfaces	7
3.2. Hardware Interfaces	7
3.3. Software Interfaces	7
3.4. Communications Interfaces	7
4. System Features	8
4.1. Data Acquisition System	8
4.2. Model Training Pipeline	8
4.3. Forecasting and Inference Engine	8
5. Other Nonfunctional Requirements	8
5.1. Performance Requirements	8
5.2. Safety Requirements	9
5.3. Security Requirements	9
5.4. Software Quality Attributes	10
5.5. Business Rules	10
6. Other Requirements	10
7. Software Design	11
7.1 System Architecture	11
7.2 Detailed Design	11
7.3 Database Schema	13
7.4 Security Design	19
8. Traceability	20
Appendix A: Glossary	21
Appendix B: Analysis Models	21
Appendix C: To be determined list	21

Revision History

[illegible]

1. Introduction

1.1. Purpose

The purpose of this document is to specify the software requirements for the **Delhi Power Demand AI** system, version 1.0. This system provides high-precision electricity demand forecasting for the Delhi region using advanced Temporal Fusion Transformer (TFT) architectures. This SRS covers the entire system, including data pipelines, training modules, inference engines, and the web-based visualization frontend.

1.2. Document Conventions

- This document follows a standard markdown structure.
- Requirements are organized by features and assigned priorities (High, Medium, Low).
- Specific placeholders like [TO BE FILLED BY USER] indicate areas requiring user-specific organizational or personal details.
- Standard SI units (MW for power, °C for temperature) are used throughout.

1.3. Intended Audience and Reading Suggestions

The intended audience for this SRS includes: -

Developers: For implementation and maintenance of the ML models and web application.

Power Grid Operators: To understand the system's predictive capabilities and limitations.

System Administrators: For deployment and configuration details.

Energy Analysts: To leverage the forecasting results for grid management. It is suggested to read Section 2 (Overall Description) for a high-level overview before proceeding to the detailed requirements in Sections 3 and 4.

1.4. Product Scope

The Delhi Power Demand AI is an end-to-end, production-grade forecasting system designed to support real-time and strategic grid operations. It automates the continuous ingestion of historical and live power load data from SLDC alongside high-resolution weather variables sourced from

Open-Meteo. Using this multi-source data pipeline, the system trains, validates, and deploys deep learning models capable of generating probabilistic power demand forecasts at 15-minute intervals for up to 48 hours ahead.

Objective: Improve grid stability and operational planning by delivering accurate, uncertainty-aware power demand forecasts through a modular AI-based system designed for seamless integration into existing and future smart grid infrastructures.

Benefits: Reduce operational costs by improving demand–supply alignment, optimizing generation and reserve scheduling, and enabling proactive load management through accurate, uncertainty-aware power demand forecasting.

1.5. References

1. Delhi State Load Dispatch Centre (SLDC) - <https://www.delhisldc.org>
2. Open-Meteo API Documentation - <https://open-meteo.com>
3. PyTorch Forecasting Library
4. PyTorch Lightning Framework
5. Streamlit Framework Documentation

2. Overall Description

2.1. Product Perspective

The Delhi Power Demand AI is a modular, AI-driven forecasting subsystem designed to operate as part of a larger power system analytics and decision-support framework rather than as an isolated application. It modernizes existing fragmented and manual forecasting practices by providing an automated, data-driven alternative that integrates real-time and historical data processing, model training, and forecast deployment within a unified pipeline. The system interfaces externally with the Delhi SLDC web portal to ingest near–real-time load data and with the Open-Meteo API to obtain historical and forecast weather parameters, which serve as critical exogenous inputs for demand prediction. Its architecture supports future integration with grid operation platforms such as EMS and SCADA systems, enabling seamless adoption within evolving smart grid infrastructures.

2.2. Product Functions

The major functions of the product include: -

Automated Data Fetching: Scrapes SLDC and queries Open-Meteo.

Feature Engineering: Calculates sinusoidal time features, holiday flags, and lag variables.

Model Training: Implements Temporal Fusion Transformer (TFT) training with automatic checkpointing.

Recursive Forecasting: Generates multi-step ahead predictions (96-192 steps).

Interactive Dashboard: Provides a UI for viewing forecasts, actuals, and accuracy metrics.

Scenario Simulation: Allows users to adjust temperature and load scaling to see “what-if” impacts.

2.3. User Classes and Characteristics

- **ML Engineers (Technical Users):** High expertise; requirement for detailed training logs and checkpoint management.
- **Grid Operators (Operational Users):** High domain knowledge; requirement for clear, actionable forecast curves and peak time alerts.
- **Energy Analysts (Analytical Users):** Moderate technical expertise; preference for data export and historical comparison.

2.4. Operating Environment

- **Hardware:** GPU with minimum 6GB VRAM (required for training).
- **OS:** Linux (Optimized for Debian/Ubuntu distributions)/ Windows.
- **Python Runtime:** Version 3.9 or higher.
- **Web Browser:** Modern browsers (Chrome, Firefox, Safari) for Streamlit frontend.

2.5. Design and Implementation Constraints

- **Timing Requirements:** Forecast generation must be fast enough for operational use (< 30s).
- **Technology Stack:** Must use PyTorch/PyTorch Lightning as per the existing implementation for TFT compatibility.
- **Data Resolution:** Fixed at 15-minute intervals (non-negotiable as per SLDC standards).

2.6. User Documentation

- **README.md:** Initial configuration and setup guide.
- **Inline Help:** Tooltips provided within the Streamlit interface.

2.7. Assumptions and Dependencies

- **Assumption 1:** The SLDC website structure remains stable enough for parsing.
- **Assumption 2:** Internet connectivity is available for real-time data fetching.
- **Dependency:** Relies on the availability of the Open-Meteo API service.

3. External Interface Requirements

3.1. User Interfaces

The user interface is a web-based dashboard built using Streamlit, designed for intuitive and efficient interaction by grid operators and analysts. It features a sidebar for user controls such as date selection and forecast adjustments, and a main panel for displaying key metrics and visualizations. A premium dark-mode design with glassmorphism effects enhances readability and reduces visual fatigue.

Interactive Recharts charts display probabilistic demand forecasts using P10, P50 (median), and P90 quantiles, enabling users to easily interpret uncertainty, compare scenarios, and support proactive load management and planning decisions.

3.2. Hardware Interfaces

- **GPU Interface:** Interface with NVIDIA drivers via CUDA 11+ for accelerated tensor computations.
- **Storage:** Minimum 10GB persistent storage for data Parquet files and checkpoints.

3.3. Software Interfaces

- **Database:** Local Parquet file-based data storage (data/).
- **SLDC Interface:** HTTP-based web scraper targeting delhisldc.org.
- **Open-Meteo API:** REST API interface via HTTPS for JSON weather responses.
- **PyTorch/TFT:** Core ML library interface for model loading and inference.

3.4. Communications Interfaces

- **HTTPS & WebSockets:** Facilitates secure, low-latency communication between the Streamlit server and the client browser.
- **TLS Encryption:** Protects data in transit, ensuring confidentiality and message integrity.
- **Environment Variables:** Secures sensitive configurations and model paths by decoupling them from the source code.

4. System Features

4.1. Data Acquisition System

4.1.1 Description and Priority Fetches raw load and weather data. **Priority: High.**

4.1.2 Stimulus/Response Sequences -

Stimulus: User clicks “Run Forecast” or scheduled task starts.

Response: System fetches SLDC and Open-Meteo data, aligns them, and saves to cache.

4.1.3 Functional Requirements -

REQ-DAQ-1: Fetch hourly weather data and interpolate to 15-min intervals. -

REQ-DAQ-2: Scrape 15-min SLDC load data with retry logic for network failures.

4.2. Model Training Pipeline

4.2.1 Description and Priority Trains the TFT model on historical data. **Priority: Medium.**

4.2.2 Functional Requirements -

REQ-TRN-1: Support training from scratch or resuming from last.ckpt.

REQ-TRN-2: Save all training checkpoints to ensure no model states are lost.

4.3. Forecasting and Inference Engine

4.3.1 Description and Priority Generates future demand predictions. **Priority: High.**

4.3.3 Functional Requirements -

REQ-INF-1: Generate probabilistic predictions for P10, P50, and P90 quantiles.

REQ-INF-2: Support recursive multi-day forecasting up to 48 hours.

5. Other Nonfunctional Requirements

5.1. Performance Requirements

- **Latent Inference:** The system shall generate a complete 24-hour power demand forecast at 15-minute resolution within 30 seconds to support near-real-time grid planning and operational

decision-making.

- **Memory Usage:** Model training shall be constrained to operate within a 6 GB VRAM GPU using a batch size of 256 when trained locally.
- **Consistency:** Forecast inference performance shall remain stable across different forecast horizons and input weather conditions.
- **Scalability:** Performance constraints shall be maintained without degradation as historical data volume increases.
- **Deployment Feasibility:** The system shall meet all performance requirements on standard development or deployment hardware.

5.2. Safety Requirements

- **Data Integrity:** The system shall ensure that historical load and weather data is not lost or corrupted during incremental data updates.
- **Safe Update Mechanism:** Incremental updates shall be performed in a manner that prevents overwriting or partial data writes in case of failures.
- **Fallback Handling:** The system shall provide dummy or cached fallback data when external APIs are unreachable.
- **UI Stability:** Fallback mechanisms shall prevent user interface crashes and ensure continued dashboard availability.
- **Graceful Degradation:** The system shall continue to operate in a limited but stable mode during data source or network failures.

5.3. Security Requirements

- **Secrets Management:** Sensitive information such as API keys, file paths, and environment-specific parameters shall be stored in a .env file.
- **Version Control Protection:** The .env file and other sensitive configurations shall be excluded from version control systems to prevent accidental exposure.
- **Local Data Storage:** Power usage and operational data shall be stored locally and not transmitted to unauthorized external services.

- **Data Privacy Compliance:** Data storage and handling practices shall comply with applicable government and regulatory guidelines.
- **Unauthorized Access Prevention:** Access to sensitive configuration files and data shall be restricted to authorized users only.

5.4. Software Quality Attributes

- **Maintainability:** The system shall use a modular code structure with clear separation between backend logic and frontend components.
- **Ease of Modification:** Modular design shall allow individual components to be updated or replaced without impacting the entire system.
- **Code Readability:** Clear module boundaries shall improve code readability and simplify debugging and testing.
- **Reliability:** Automated checkpointing mechanisms shall be implemented during model training.
- **Failure Recovery:** Checkpoints shall ensure that training progress can be resumed after interruptions without permanent loss.

5.5. Business Rules

- Only authorized users with verified access to the server or execution environment shall be permitted to initiate model training or retraining processes.
- Authorization shall be enforced at the system or environment level to prevent unauthorized triggering of compute-intensive training tasks.
- Training permissions shall be restricted to prevent accidental or malicious misuse of system resources.
- Model training operations shall be executed in a controlled manner to ensure system stability and reproducibility.

6. Other Requirements

- **Containerized Deployment:** The system shall support deployment using Docker containers to ensure consistent behavior across different operating systems and environments.

- **Configuration Flexibility:** System configurations such as data paths, model parameters, and runtime settings shall be externally configurable without requiring code changes.
- **Portability:** The system shall be portable across local development setups, institutional servers, and cloud-based infrastructure.
- **Logging and Monitoring:** Basic logging mechanisms shall be provided to record system execution, errors, and training events for debugging and maintenance purposes.

7. Software Design

7.1. System Architecture

- The system follows a modular, layered architecture that separates the user interface, backend processing, model logic, and data storage components.
- A web-based presentation layer provides user interaction, forecast visualization, and scenario adjustments, without direct access to backend data sources or model internals.
- The backend processing layer handles data ingestion, preprocessing, model training, and inference through dedicated functional modules.
- The model layer encapsulates the Temporal Fusion Transformer (TFT) used for probabilistic power demand forecasting.
- A local storage layer is used to persist historical data, trained model checkpoints, and forecast outputs.

7.2. Detailed Design

The internal design of the Delhi Power Demand AI system is organized around modular components that handle data acquisition, preprocessing, model training, forecasting, and visualization. These components are designed with clear responsibilities and well-defined interaction boundaries to support maintainability and ease of extension. The design captures both the static structure of the system and the dynamic behavior during execution. Class diagrams are used to represent the logical organization of system components, while sequence diagrams illustrate the interactions between components during key workflows such as data updates, model training, and forecast generation.

7.2.1. Class diagrams

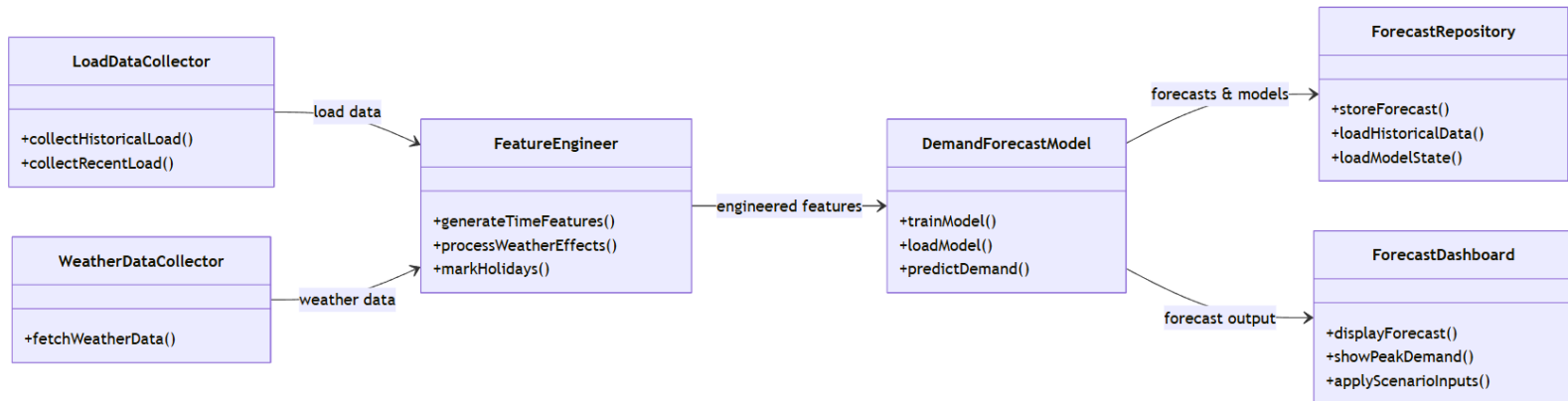
The class diagram illustrates the static structure of the Delhi Power Demand AI system by identifying the main software components and their responsibilities. The design follows a modular, pipeline-oriented approach in which each class performs a specific role in the demand forecasting process.

The **ForecastDashboard** class represents the user-facing component responsible for displaying demand forecasts, peak demand information, and scenario-based analysis results. It interacts with the forecasting model without directly accessing data sources.

The **LoadDataCollector** and **WeatherDataCollector** classes are responsible for acquiring historical and recent electricity load data and weather parameters respectively, reflecting the strong dependence of Delhi's load profile on weather conditions.

The **FeatureEngineer** class processes raw load and weather data to generate time-based features, encode weather effects, and mark holidays, preparing structured inputs for the forecasting model.

The **DemandForecastModel** class encapsulates the AI-based forecasting logic used to train the model and generate future electricity demand and peak demand projections.



7.2.1.1. Weather Data Acquisition and Interpolation

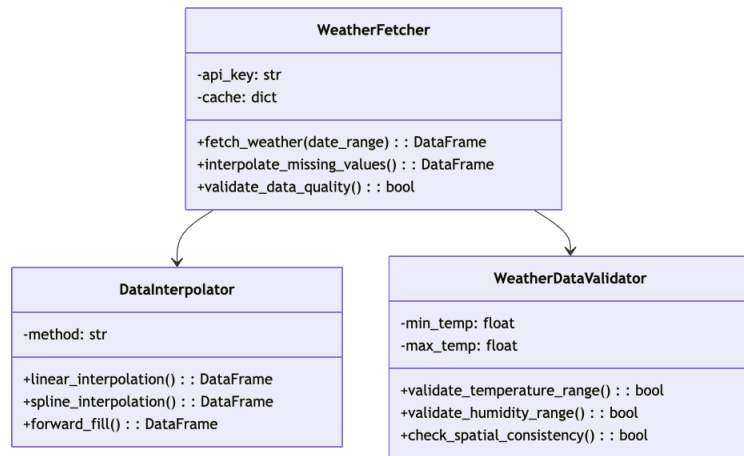
This module is responsible for fetching, validating, and preprocessing weather data from external APIs. It ensures that missing datetimes are handled accurately using advanced interpolation techniques to provide a continuous time-series dataset for the forecasting models.

Class Specifications:

WeatherFetcher: Manages API connections, handles requests to the external weather service, and caches responses to minimize redundant network calls.

DataInterpolator: Implements algorithms (linear, spline, forward fill) to estimate missing weather variables (e.g., temperature, humidity) in the time series.

WeatherDataValidator: Enforces physical constraints on the data, ensuring values fall within realistic meteorological bounds and maintaining spatial/temporal consistency.

**D-CL-02:** Class diagram for data scraper

7.2.1.2. SLDC Load Data Scraping

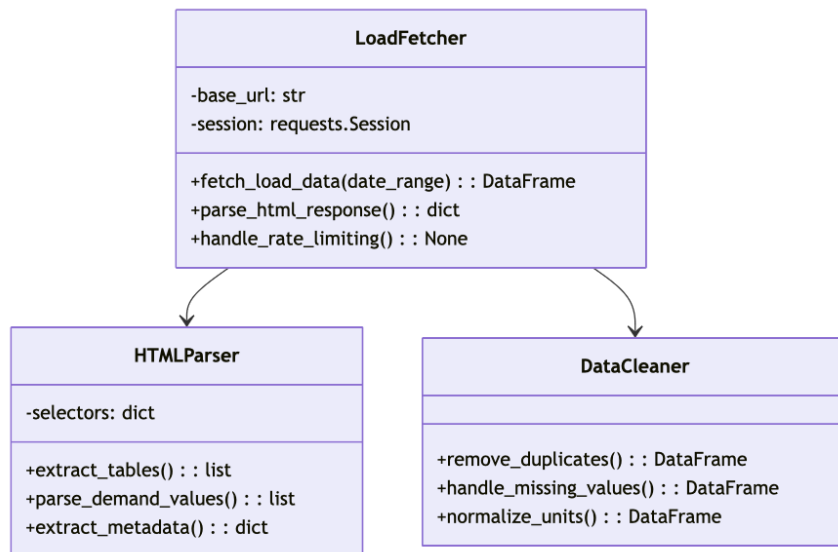
This module performs automated data retrieval from the State Load Despatch Center (SLDC) portals. It parses complex HTML tables to extract historical demand load data, normalizes the units, and cleans the dataset before ingestion into the data warehouse.

Class Specifications:

LoadFetcher: Manages HTTP sessions, executes the web scraping requests over specific date ranges, and handles rate-limiting to prevent server blocking.

HTMLParser: Uses CSS selectors and DOM traversal to extract raw tabular data and metadata from the SLDC response payload.

DataCleaner: Sanitizes the extracted data by removing duplicates, resolving missing metrics, and normalizing power units to a standard format (e.g., MW).

**D-CL-02:** Class diagram for data scraper

7.2.1.3. Model Training and Checkpoint Preservation

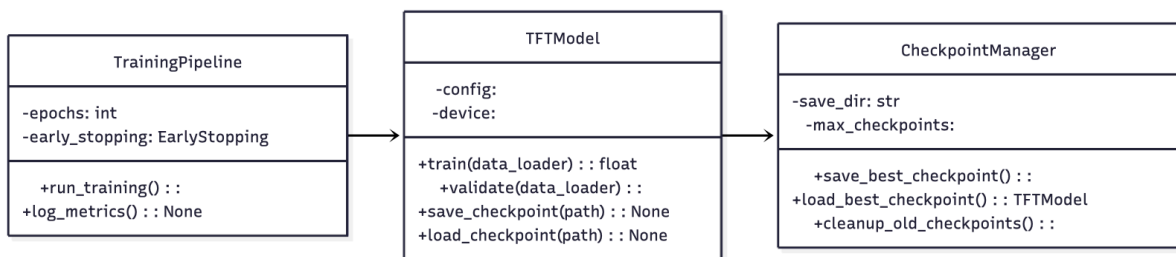
This subsystem orchestrates the training lifecycle of the Temporal Fusion Transformer (TFT) model. It ensures resilience against interruptions by preserving intermittent weights (checkpoints) and maintaining historical training records to resume halted operations seamlessly.

Class Specifications:

TFTModel: Encapsulates the PyTorch Lightning/Neural Network logic, handling forward passes, loss calculation, and model weight persistence.

CheckpointManager: Monitors storage capacity, enforces retention policies for saved checkpoints, and automatically retrieves the best-performing model weights based on validation loss.

TrainingPipeline: The primary controller that defines the epoch loops, triggers early stopping, and logs hyperparameter metrics.



D-CL-04: Class Diagram for Model Training

7.2.1.4. Probabilistic Demand Forecasting

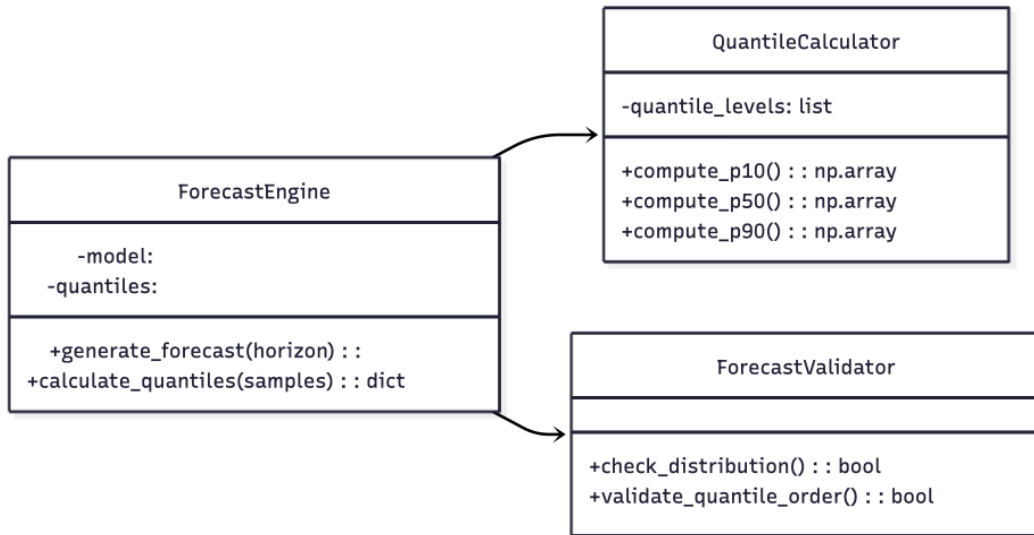
To provide actionable insights, the inference engine generates probabilistic forecasts rather than simple point predictions. It calculates the 10th (P10), 50th (P50), and 90th (P90) percentiles, giving operational teams a reliable confidence interval for grid management.

Class Specifications:

ForecastEngine: Accepts input streams, triggers model inference, and aggregates raw prediction distributions.

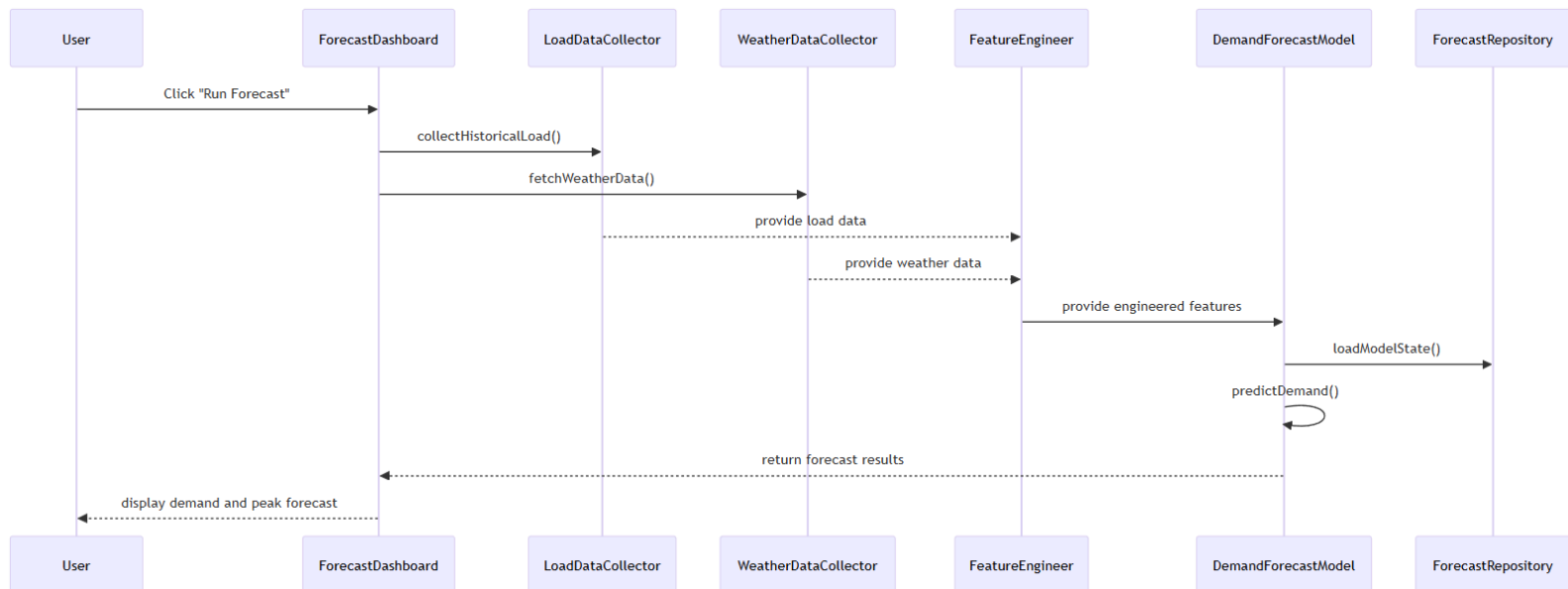
QuantileCalculator: Processes raw model output samples to compute specific statistical bounds (P10, P50, P90).

ForecastValidator: Ensures the logical integrity of the generated outputs (e.g., verifying that $P10 < P50 < P90$) before they are dispatched to the client.

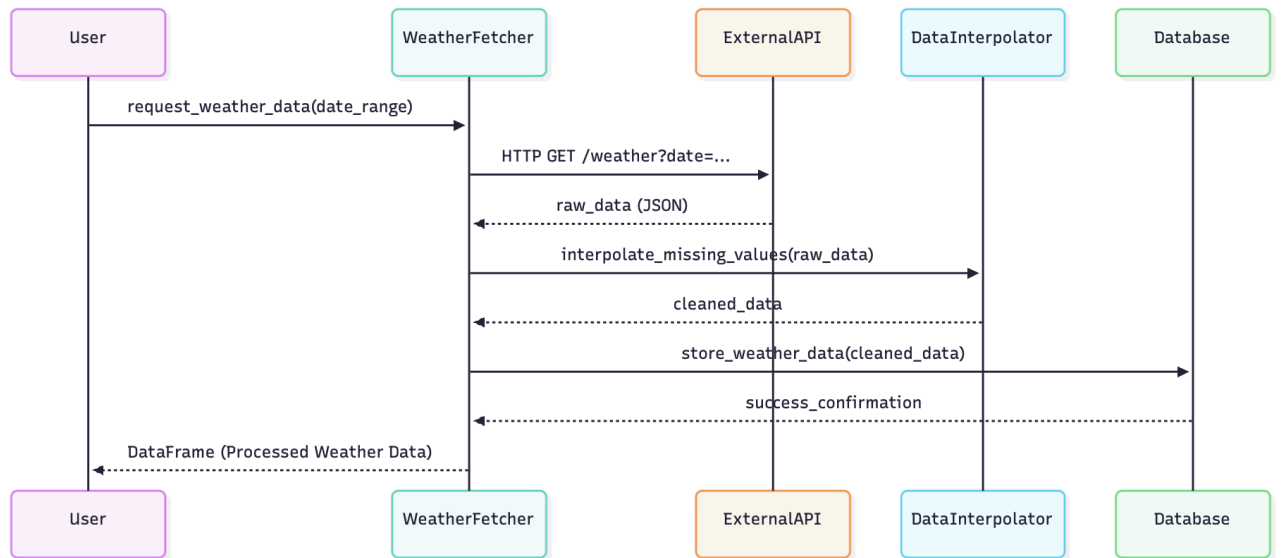
**D-CL-05:** Class diagram of Probabilistic Demand Forecasting

7.2.2. Sequence Diagrams

The sequence diagram describes the step-by-step interaction between system components during the execution of a demand forecasting operation. The workflow begins when the user triggers the *Run Forecast* action through the dashboard interface. The dashboard initiates the collection of historical electricity load data and relevant weather parameters by invoking the respective data collection components. Once the load and weather data are retrieved, they are forwarded to the feature engineering component, which prepares time-based, weather-related, and holiday features required for forecasting. These engineered features are then provided to the demand forecasting model, which loads the trained model state and computes probabilistic electricity demand and peak demand forecasts. The generated forecast results are returned to the dashboard, where they are displayed for user interpretation and analysis.

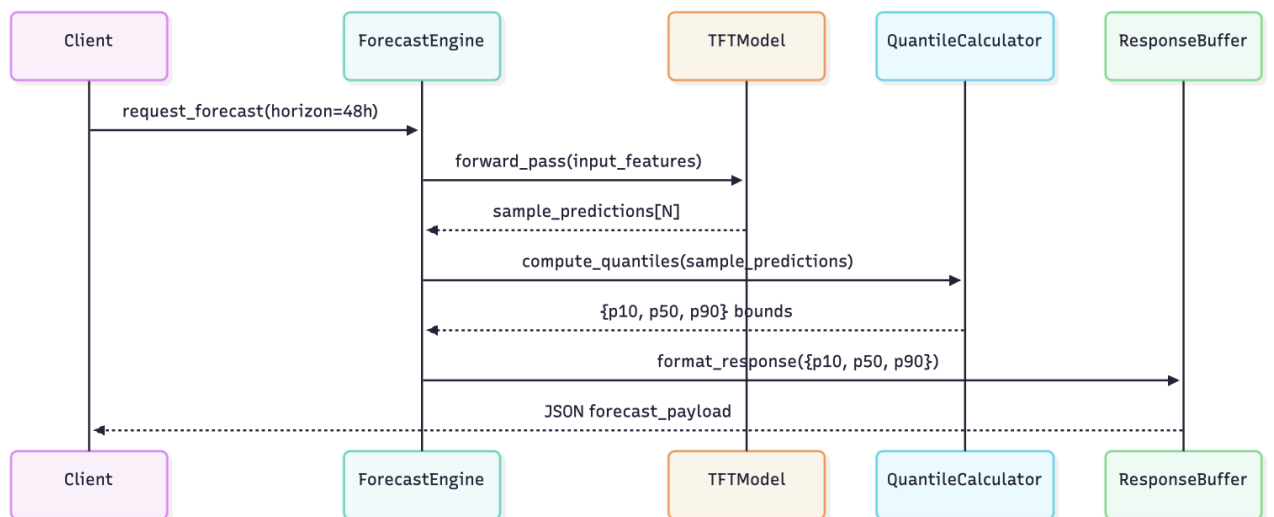


7.2.2.1. Weather Data Acquisition Flow

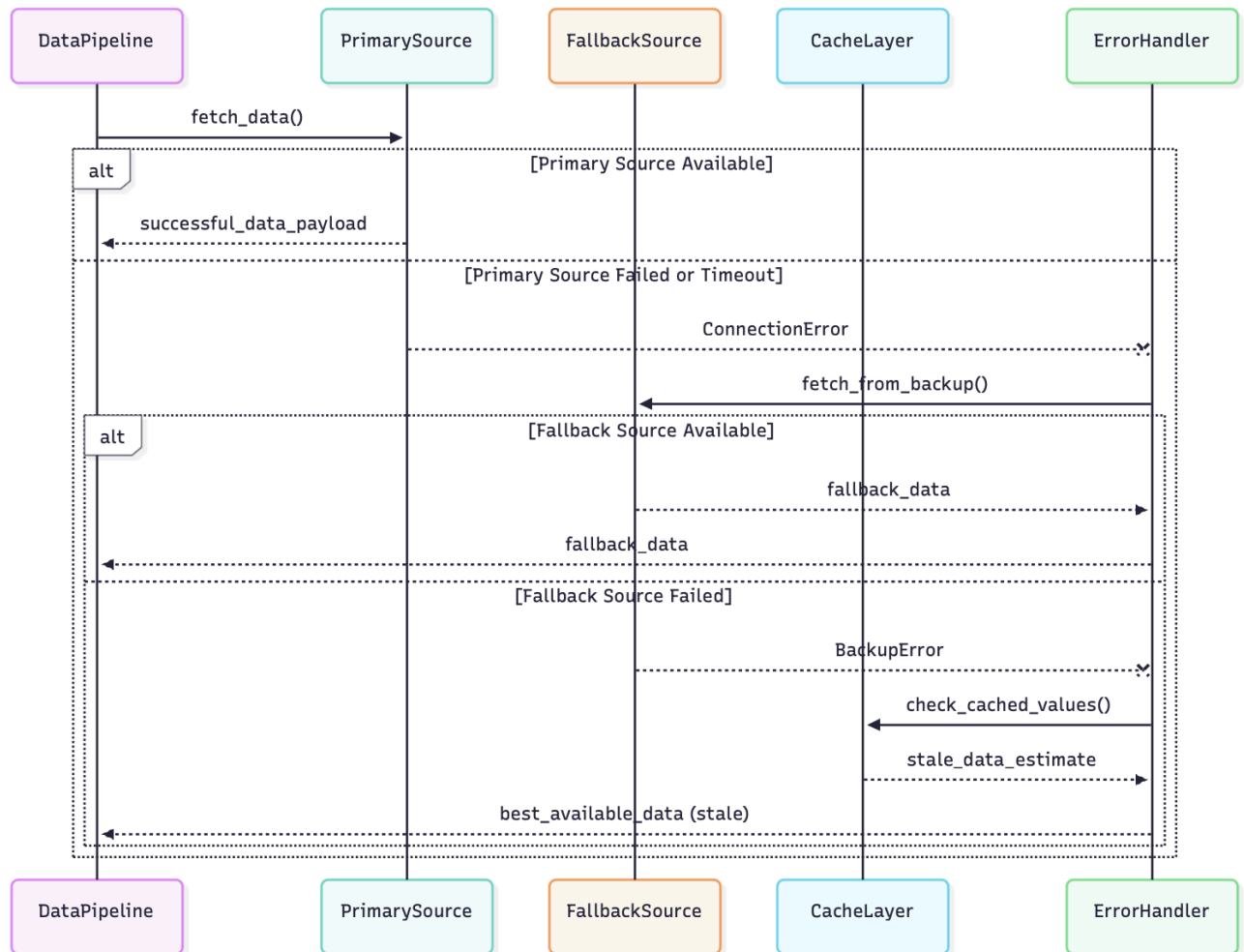


D-SQ-01: Sequence Diagram of Weather data flow

7.2.2.2. Probabilistic Forecast Generation Flow

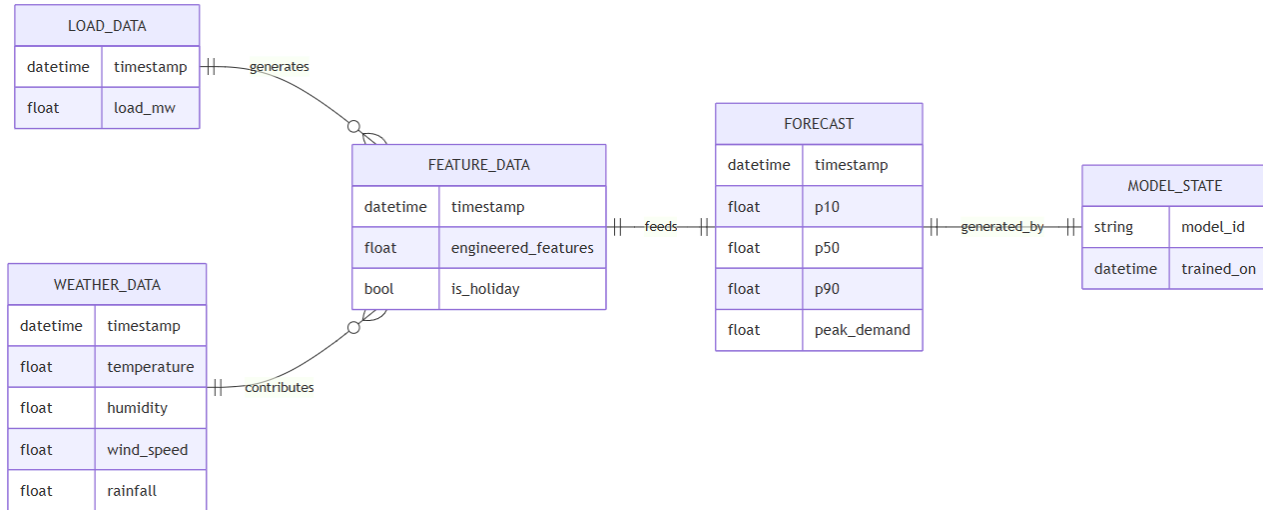


D-SQ-02: Sequence Diagram of Probabilistic Forecast Generation Flow

7.2.2.3. Fallback Handling for Data Unavailability**D-SQ-03:** Sequence Diagram of Probabilistic Forecast Generation Flow

7.3. Database Schema

The Entity Relationship Diagram (ERD) represents the logical organization of data used within the Delhi Power Demand AI system. It identifies the key data entities involved in electricity demand forecasting and illustrates the relationships between raw input data, engineered features, forecast outputs, and model states. The ERD focuses on data flow and dependency relationships rather than physical database implementation details, reflecting the system's file-based Parquet storage design.



D-DB-01: Database schema of Historical Data Integrity

7.4. Security Design

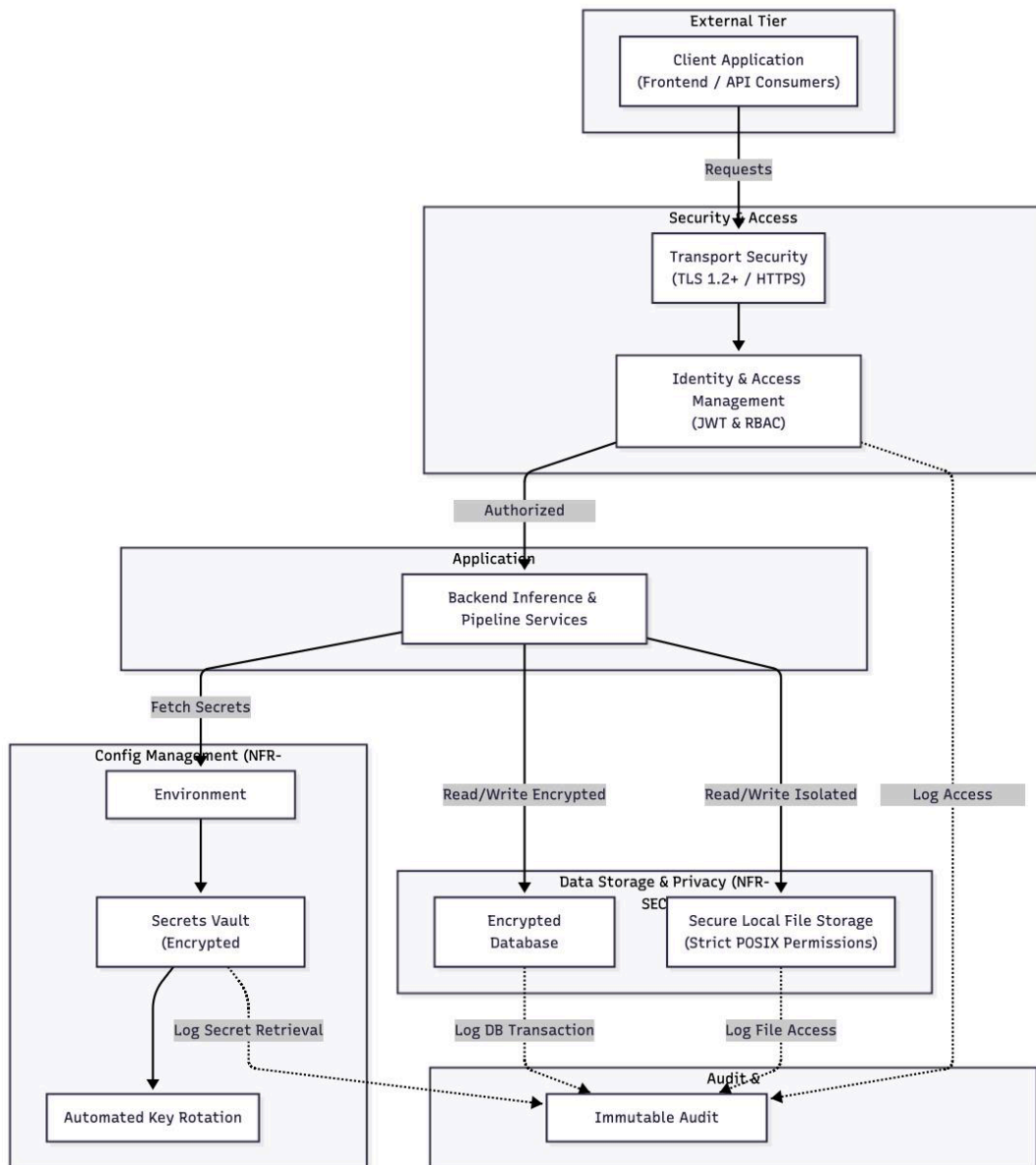
The security design of the Delhi Power Demand AI system focuses on protecting sensitive configuration information, restricting access to critical operations, and ensuring safe interaction with external data sources. As the system is designed for analytical and forecasting purposes rather than public access, security controls are primarily enforced at the application and environment levels. The design prioritizes simplicity, data confidentiality, and controlled system usage over complex authentication frameworks.

7.4.1. Authentication and Authorization Mechanisms

- The system does not implement end-user authentication within the application interface, as it is intended to operate in a controlled server or institutional environment.
- Access to system execution, configuration, and model training is restricted to authorized users with access to the server or runtime environment.
- Authorization to trigger computationally intensive operations such as model training or retraining is enforced at the operating system or deployment level.
- This approach prevents unauthorized usage of system resources and ensures controlled modification of forecasting models.

7.4.2. Security Protocols

- Sensitive configuration details such as API keys, file paths, and environment-specific parameters are managed using environment variables stored in a `.env` file.
- The `.env` file and other sensitive configuration artifacts are excluded from version control systems to prevent accidental disclosure.
- External data sources are accessed using secure communication protocols (HTTPS) to ensure data integrity during transmission.
- All power demand, weather, and forecast data is stored locally, ensuring compliance with applicable data privacy and government guidelines.



Security Design

8.Traceability

Traceability establishes a clear linkage between the requirements defined in the Software Requirements Specification and the corresponding design elements presented in the Software Design section. This mapping ensures that all functional and non-functional requirements are adequately addressed by the system architecture, detailed design components, and supporting diagrams. The traceability matrix demonstrates completeness, consistency, and accountability by showing how each requirement is realized through specific design artifacts such as class diagrams, sequence diagrams, database schema, and security design.

Requirement ID	Requirement Name	Design ID(s)	Design Artifact
REQ-DAQ-1	Weather Data Acquisition and Interpolation	D-CL-01, D-SQ-01	Class Diagram, Sequence Diagram
REQ-DAQ-2	SLDC Load Data Scraping	D-CL-02	Class Diagram
REQ-TRN-1	Model Training and Resume Capability	D-CL-04	Class Diagram
REQ-TRN-2	Training Checkpoint Preservation	D-CL-04, D-DB-02	Class Diagram, Database Schema
REQ-INF-1	Probabilistic Demand Forecasting (P10, P50, P90)	D-CL-05, D-SQ-03	Class Diagram, Sequence Diagram
REQ-INF-2	Recursive Multi-Day Forecasting (up to 48 hours)	—	—
NFR-PERF-1	Forecast Latency Constraint (<30 seconds)	D-ARCH-01	System Architecture
NFR-MEM-1	GPU Memory Constraint (6GB VRAM)	—	—
NFR-SAFE-1	Historical Data Integrity	D-DB-01	Database Schema
NFR-SAFE-2	Fallback Handling for Data Unavailability	D-SQ-01	Sequence Diagram
NFR-SEC-1	Secure Configuration Management	D-SEC-01	Security Design
NFR-REL-1	Training Reliability via Checkpoints	D-CL-04	Class Diagram

Appendix A: Glossary

- **SLDC:** State Load Dispatch Centre.
- **TFT:** Temporal Fusion Transformer.
- **MW:** Megawatt.
- **Quantile:** A probabilistic interval (e.g., P90 means 90% confidence the value is below this).
- **Parquet Storage:** Columnar file-based storage format used for efficient local persistence of historical load
- **SCADA:** Supervisory Control and Data Acquisition
- **EMS:** Energy Management System
- **Model Checkpoint:** A saved snapshot of a trained model's parameters allows training to resume without losing progress.
- **Recursive Forecasting:** A forecasting approach where predicted values for earlier time steps are used as inputs to predict subsequent time steps.

Appendix B: Analysis Models

- **Data Flow:** Data Source -> Fetcher -> Processor -> Model -> Dashboard.

Appendix C: To Be Determined List

[TBD-1]: Exact threshold for “High Load” alerts.

[TBD-2]: Final retention duration for historical data storage (e.g., 1 year, 3 years, unlimited).

[TBD-3]: Backup policy for model checkpoints (local only or cloud replication).

[TBD-4]: Formal approval process for model retraining in operational environments.

[TBD-5]: Definition of acceptable forecast accuracy benchmark (e.g., MAPE threshold).